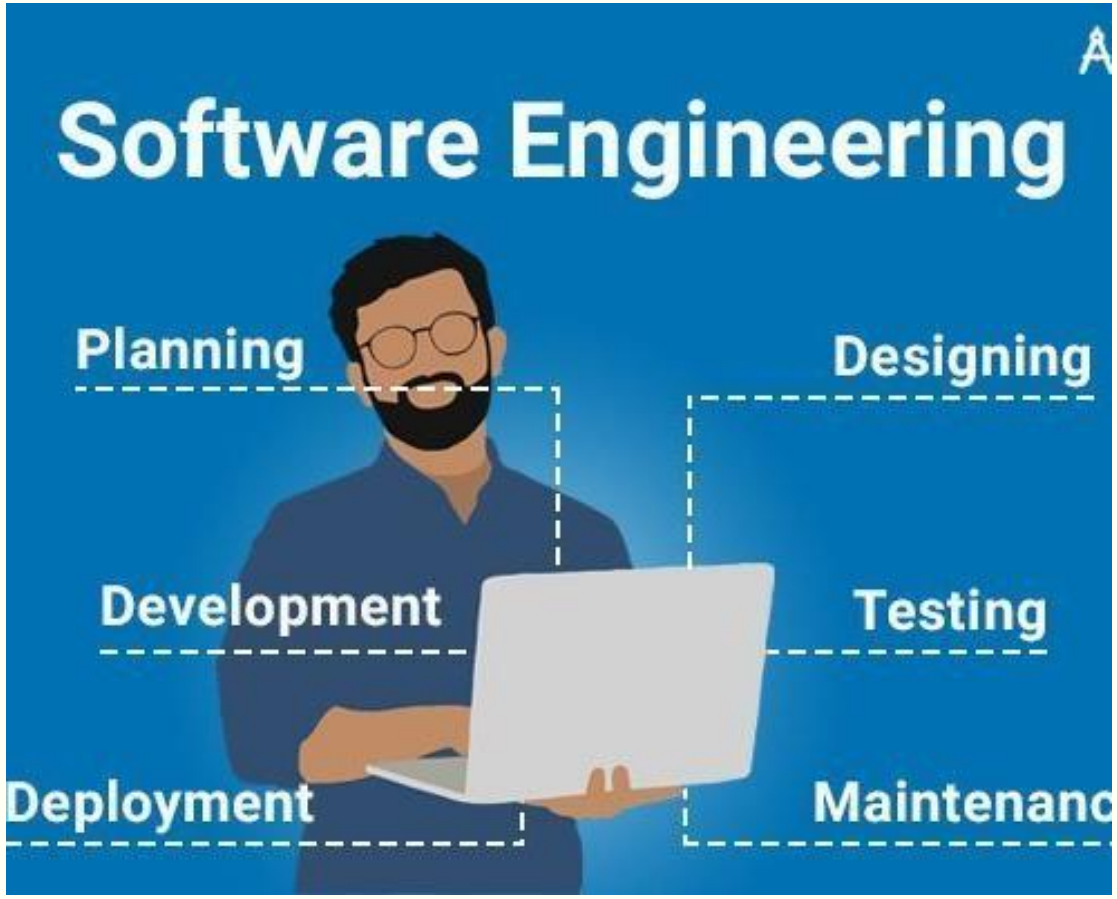
SOFTWARE ENGINEERING & PROJECT MANAGEMENT

K. K. WAGH EDUCATION SOCIETY

K K Wagh

UNIT 1

Introduction to Software Engineering and Software Process Models



Contents

- **Software Engineering :**
  - The Nature of Software, Defining Software, Software Engineering Process, Software Engineering Practice.
- **Process Models:**
  - A Generic Process Model, Process Assessment and Improvement, Prescriptive Process Models.
- **Agile Development:**
  - Agility, Agility and cost of change, Agile Process, Extreme Programming (XP), Other Agile process models.

What is Software?

- Software is:
  1. Instructions (computer programs) that when executed provide desired features, function, and performance
  2. Data structures that enable the programs to adequately manipulate information
  3. Documentation that describes the operation and use of the programs
  4. Software is both a product and a vehicle that delivers a product(e.g. Operating System)

What is Software?

- A collection of computer programs and related documents that are intended to provide desired features, functionalities and better performance.
- Software products maybe –
  - Generic: developed to be sold to a range of different customers
  - Custom: developed for a single customer according to their specification

The Nature of Software

- System software
  - Compiler, assembler, editors
- Application software
  - Stand-alone programs/systems
- Engineering/Scientific software
  - Complex numeric computations viz. astronomy, space shuttle orbital dynamics, molecular biology, system simulation etc.
- Embedded software
  - Program residing in another product or system
- Web applications
  - Web pages
- Artificial intelligence software
  - Robotics, expert systems, game playing, image and speech recognition

Characteristics of Software

1. Software is **developed** or **engineered**; it is not manufactured in the classical sense

2. Software doesn't "**wear out**"

3. Although the industry is moving toward component-based construction, most software continues to be **custom built**

Software Engineering - A Layered Technology

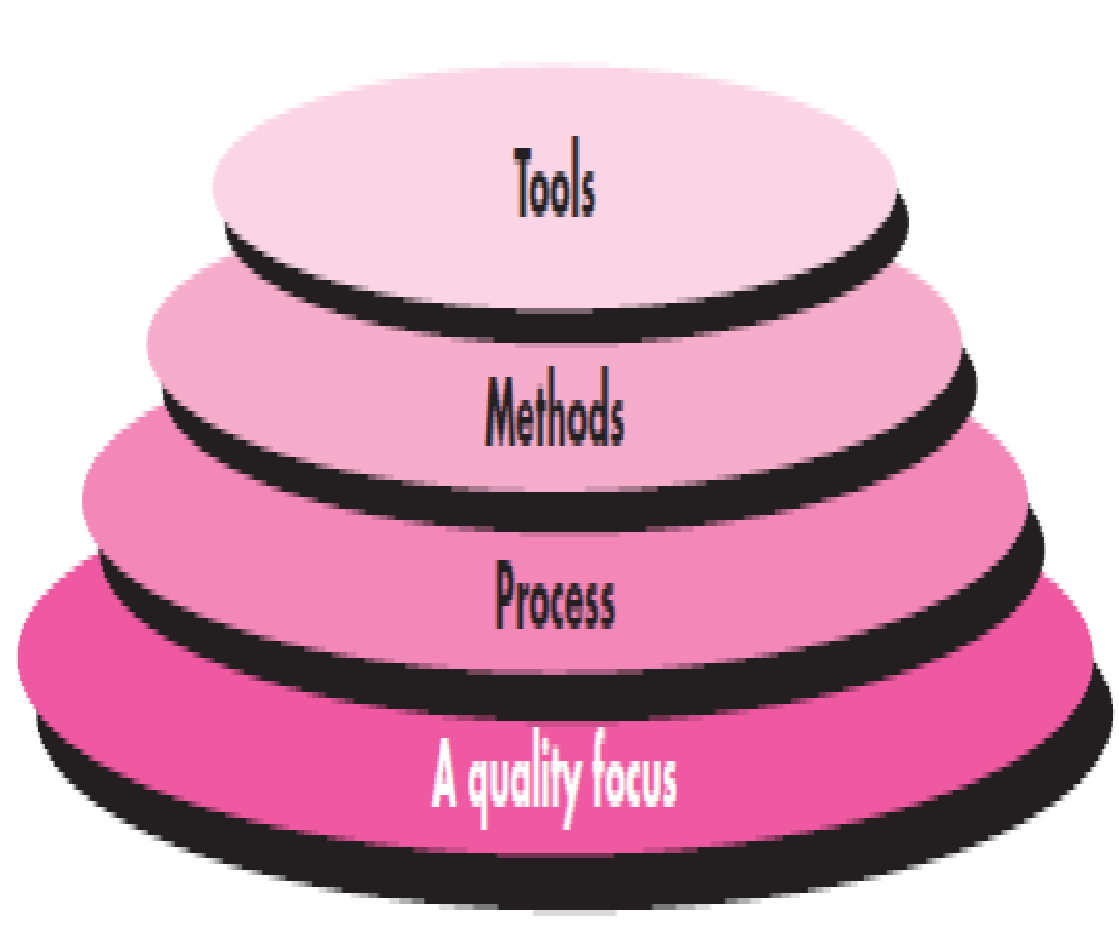


Figure: *Software Engineering Layers*

Software Engineering - A Layered Technology

- Quality: Organizational commitment to quality, Total Quality Management (TQM), Six Sigma etc.

- Process Model : A framework responsible for the context in which technical methods are applied, work products are produced, milestones are established, quality is ensured, and change is properly managed.

- Methods: A broad array of tasks that include communication, requirements analysis, design modeling, program construction, testing, and support.

- Tools : Automated or semi automated support for process and methods.

The Software Engineering Process

- Software processes in software engineering refer to the methods and techniques used to develop and maintain software.
- A process is a collection of activities, actions and tasks to create a work product.
    - An activity strives to achieve a broad objective. For example, communication with stakeholders.
    - An action encompasses a set of tasks that produce a major work product. For example, architectural design.
    - A task focuses on a small, but well-defined objective that produces a tangible outcome. For example, conducting a unit test.
- The intent is to deliver software in a timely manner with sufficient quality to satisfy end user.

The Software Engineering Process

Process Framework has 5 activities:

1. Communication

   The intent is to understand stakeholders' objectives for the project and to gather requirements that help define software features and functions.

2. Planning

   Defines the software engineering work by describing the technical tasks to be conducted, the risks that are likely, the resources that will be required, the work products to be produced, and a work schedule

3. Modeling

   A software engineer create models to better understand software requirements and the design that will achieve requirements.

4. Construction

   This activity combines code generation (either manual or automated) and the testing that is required to uncover errors in the code

5. Deployment

   The software (as a complete entity or as a partially completed increment) is delivered to the customer who evaluates the delivered product and provides feedback based on the evaluation

Umbrella Activities

Umbrella activities are applied throughout a software project and help to manage and control progress, quality, change, and risk.

- **Software project tracking and control**—allows the software team to assess progress against the project plan and take any necessary action to maintain the schedule.
- **Risk management**—assesses risks that may affect the outcome of the project or the quality of the product.
- **Software quality assurance**—defines and conducts the activities required to ensure software quality.
- **Technical reviews**—assesses software engineering work products in an effort to uncover and remove errors before they are propagated to the next activity

Umbrella Activities

- **Measurement**—defines and collects process, project, and product measures that assist the team in delivering software that meets stakeholders' needs; can be used in conjunction with all other framework and umbrella activities.
- **Software configuration management**— manages the effects of change throughout the software process.
- **Reusability management**—defines criteria for work product reuse (including software components) and establishes mechanisms to achieve reusable components.
- **Work product preparation and production** —encompasses the activities required to create work products such as models, documents, logs, forms, and lists.

Software Engineering Practice

Outline of the essence of problem solving and consequently the essence of Practice:

- Understand the problem
  communication & analysis
- Plan the solution
  modeling and software design
- Carry out the plan
  code generation
- Examine the result
  testing and quality assurance

PROCESS MODELS

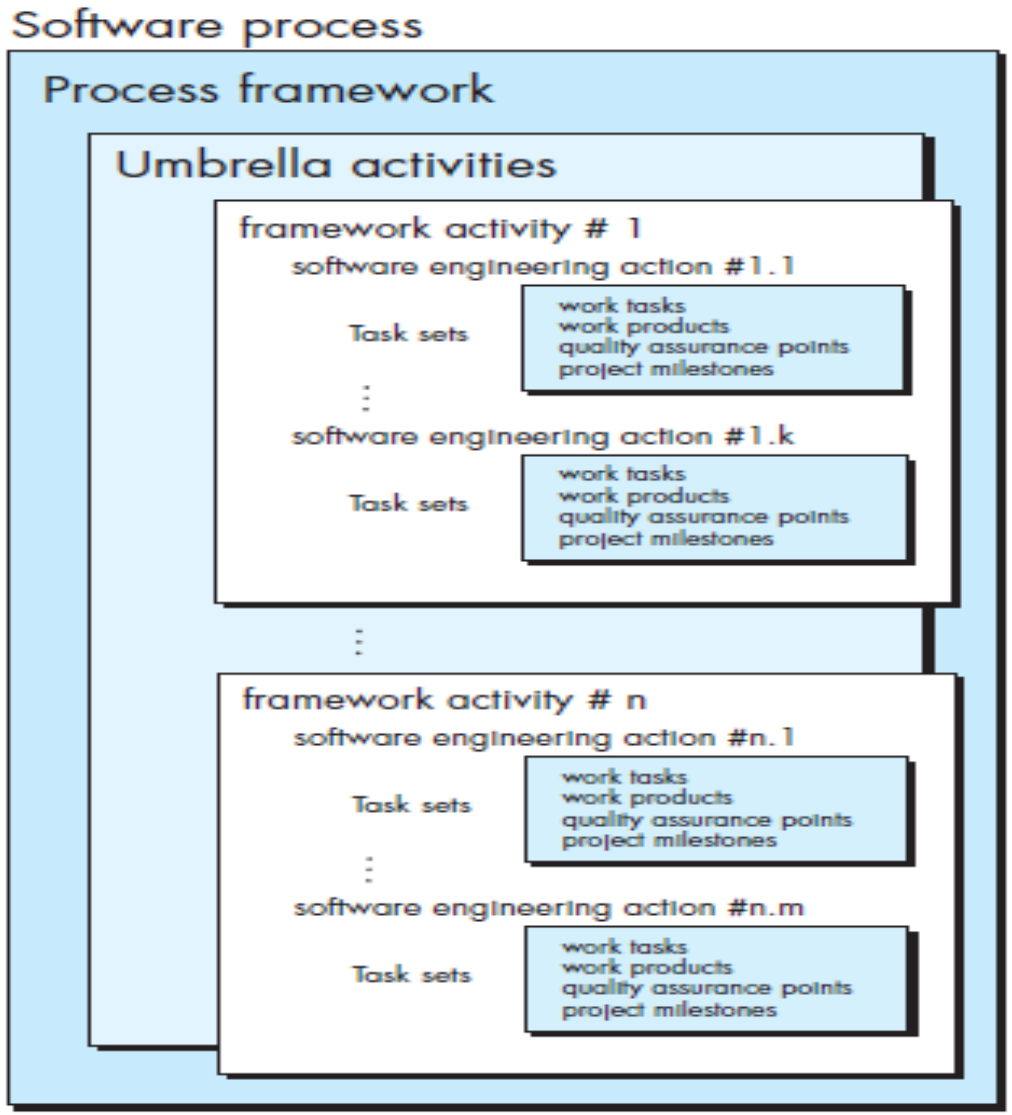
A Generic Process Model

- A process model provides a specific roadmap for software engineering work.

- Process means a series of activities taking place .

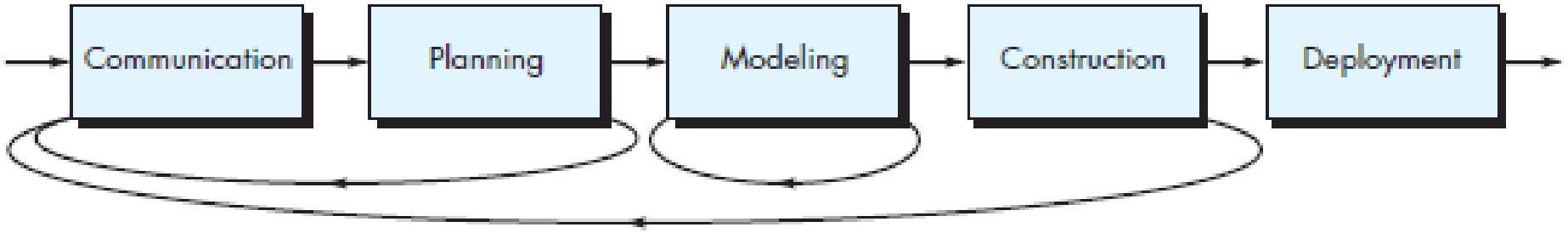
- A process was defined as a collection of work activities, actions, and tasks that are performed when some work product is to be created.

A Generic Process Models

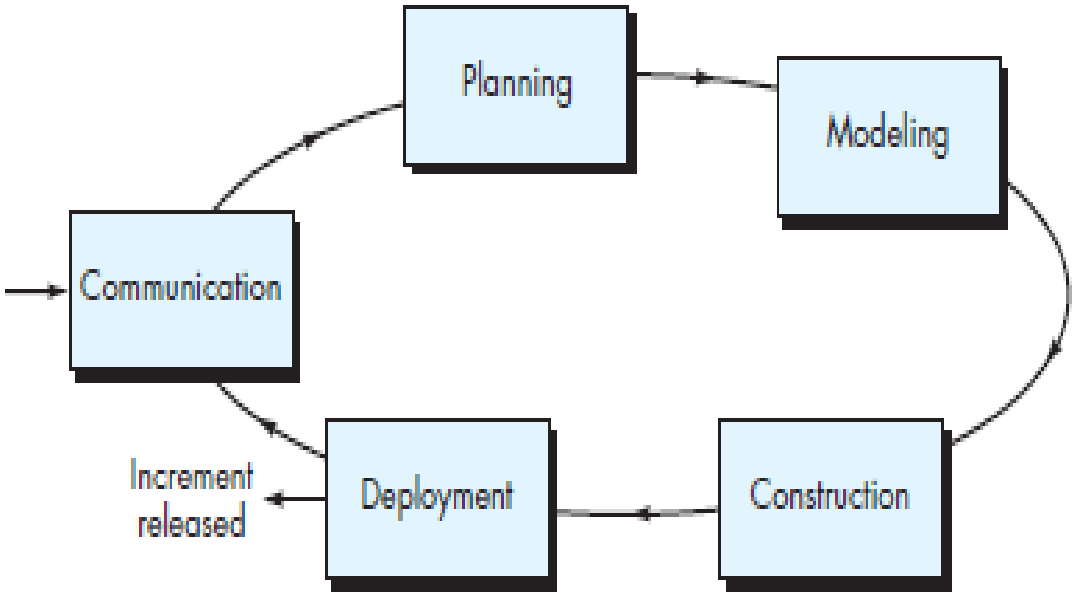


Process Flow

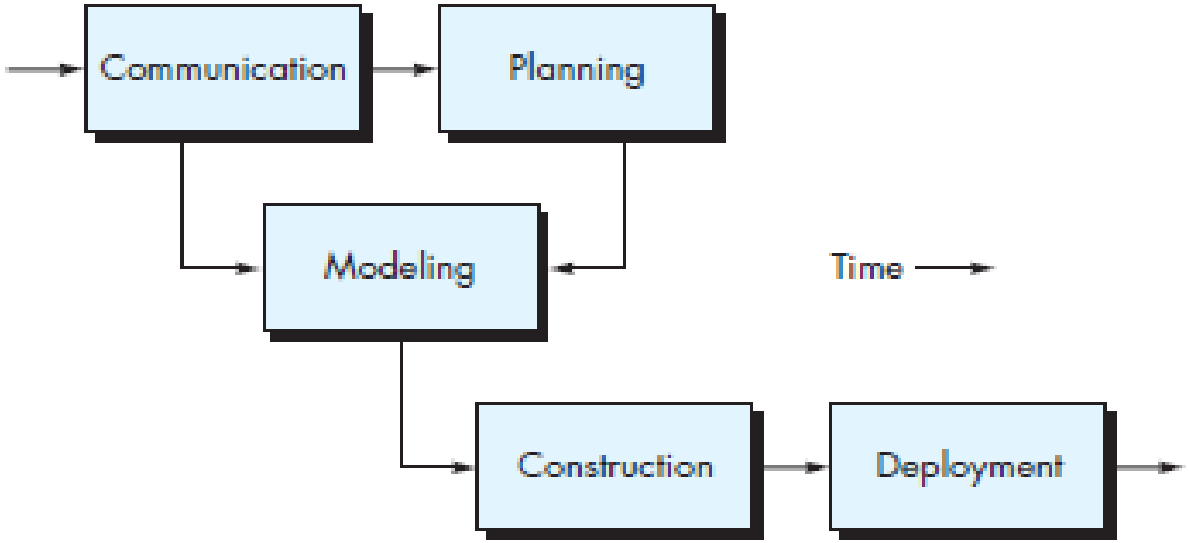
(a) Linear process flow



(b) Iterative process flow



(c) Evolutionary process flow



(d) Parallel process flow

Process Assessment and Improvement

- A number of different approaches to software process assessment and improvement have been proposed over the past few decades:

  - ➢ Standard CMMI Assessment Method for Process Improvement (SCAMPI) — provides a five step process assessment model that incorporates five phases: initiating, diagnosing, establishing, acting and learning.

  - ➢ CMM-Based Appraisal for Internal Process Improvement (CBA IPI)—provides a diagnostic technique for assessing the relative maturity of a software organization; uses the SEI CMM as the basis for the assessment

Process Assessment and Improvement

- SPICE—The SPICE (ISO/IEC15504) standard defines a set of requirements for software process assessment. The intent of the standard is to assist organizations in developing an objective evaluation of the efficacy of any defined software process.

- ISO 9001:2000 for Software—a generic standard that applies to any organization that wants to improve the overall quality of the products, systems, or services that it provides. Therefore, the standard is directly applicable to software organizations and companies.

Prescriptive Process Models

- A prescriptive process model is a model that describes "how to do" according to a certain software process system.
- A prescriptive model prescribes how a new software system should be developed.
- Prescriptive models are used as guidelines or frameworks to organize and structure how software development activities should be performed, and in what order.

Types of Prescriptive Process Models

- Waterfall model
- Incremental Model
- Evolutionary Process models
  - Prototyping Model
  - Spiral Model
- Concurrent Models

Waterfall model

- The waterfall model is also called as **'Linear sequential model'** or **'Classic life cycle model'.**
- A **systematic**, **sequential approach** to software development that begins with customer specification of requirements and progresses through planning, modeling, construction, and deployment
- Each phase is fully completed before the beginning of the next phase.
- The model is used for the small projects.
- In this model, feedback is taken after each phase to ensure that the project is on the right path.
- Testing part starts only after the development is complete.

Waterfall Model

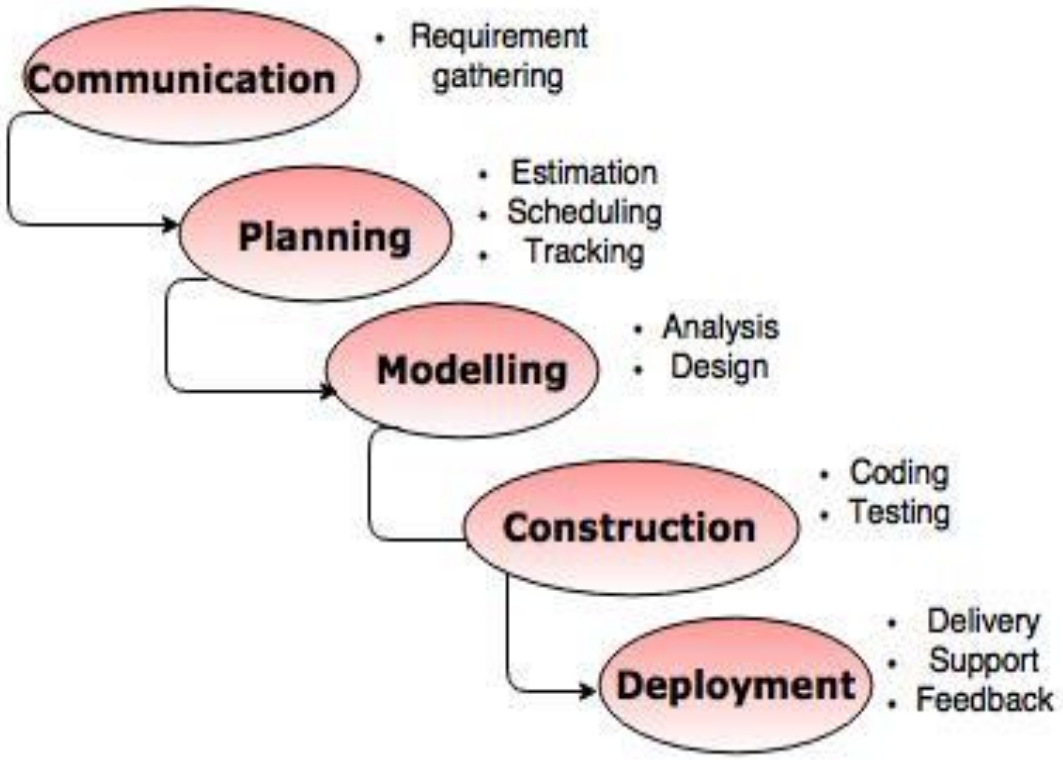
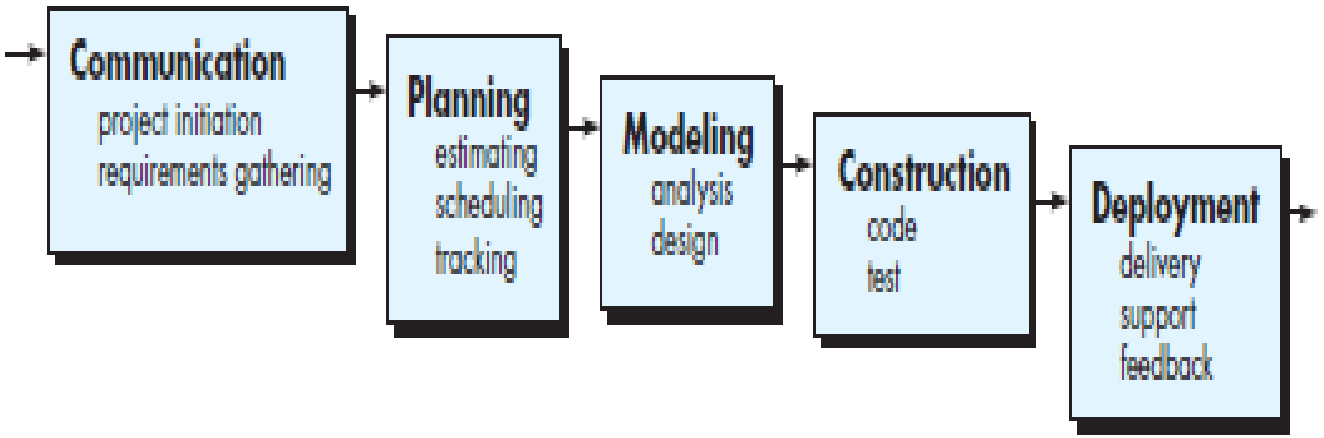


Fig. - The Waterfall model



Advantages of waterfall model

- The waterfall model is simple and easy to understand, implement, and use.
- All the requirements are known at the beginning of the project, hence it is easy to manage.
- It avoids overlapping of phases because each phase is completed at once.
- This model works for small projects because the requirements are understood very well.

Disadvantages of waterfall model

- This model is not good for complex and object oriented projects.
- The problems with this model are uncovered, until the software testing.
- The amount of risk is high.

Incremental Process model

- The incremental model combines the elements of waterfall model and they are applied in an iterative fashion.
- The first increment in this model is generally a core product.
- Each increment builds the product and submits it to the customer for any suggested modifications.
- The next increment implements on the customer's suggestions and add additional requirements in the previous increment.
- This process is repeated until the product is finished. **For example,** the word-processing software is developed using the incremental model.

Incremental Process model

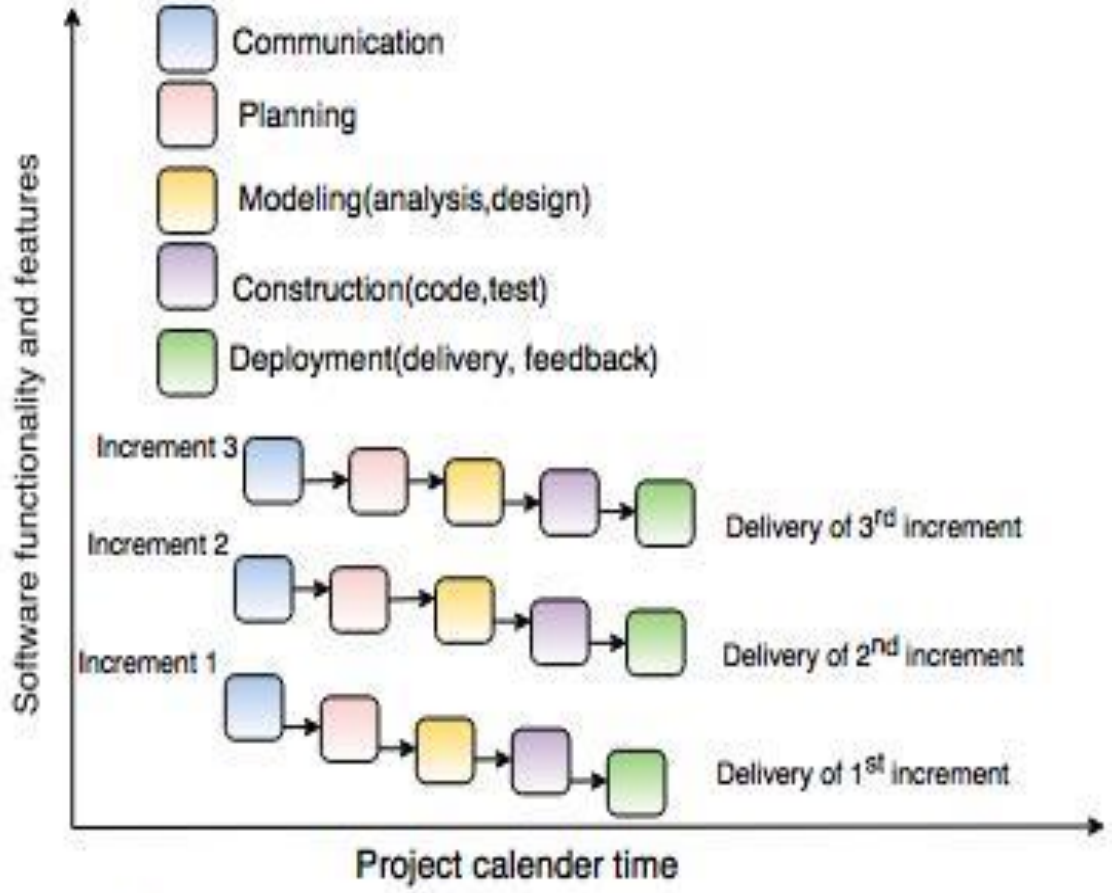


Fig. - Incremental Process Model

Advantages of incremental model

- This model is flexible because the cost of development is low and initial product delivery is faster.
- It is easier to test and debug during the smaller iteration.
- The working software generates quickly and early during the software life cycle.
- The customers can respond to its functionalities after every increment.

Disadvantages of incremental model

- The cost of the final product may cross the cost estimated initially.
- This model requires a very clear and complete planning.
- The planning of design is required before the whole system is broken into small increments.
- The demands of customer for the additional functionalities after every increment causes problem during the system architecture.

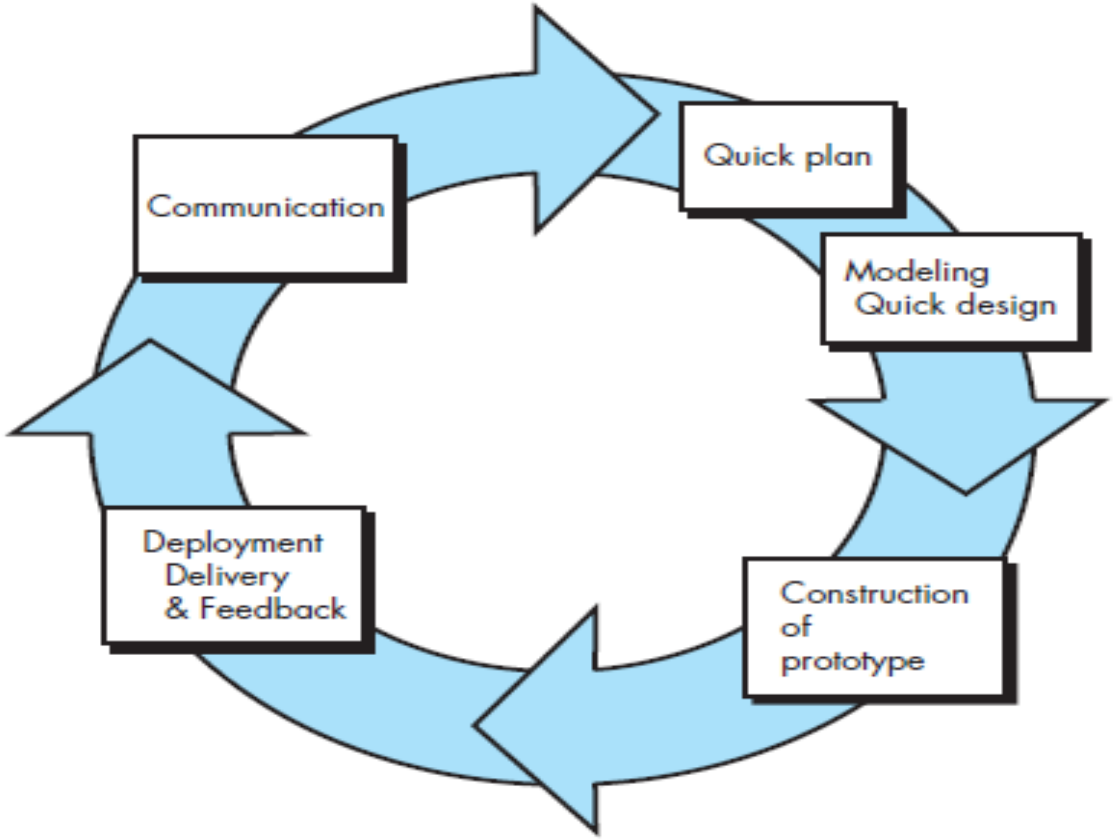
Evolutionary Process models

- Evolutionary models are iterative.
- They are characterized in a manner that enables you to develop increasingly more complete versions of the software.
- This model is also known as Successive model .
- Two common evolutionary process models are
  - Prototyping
  - The Spiral Model

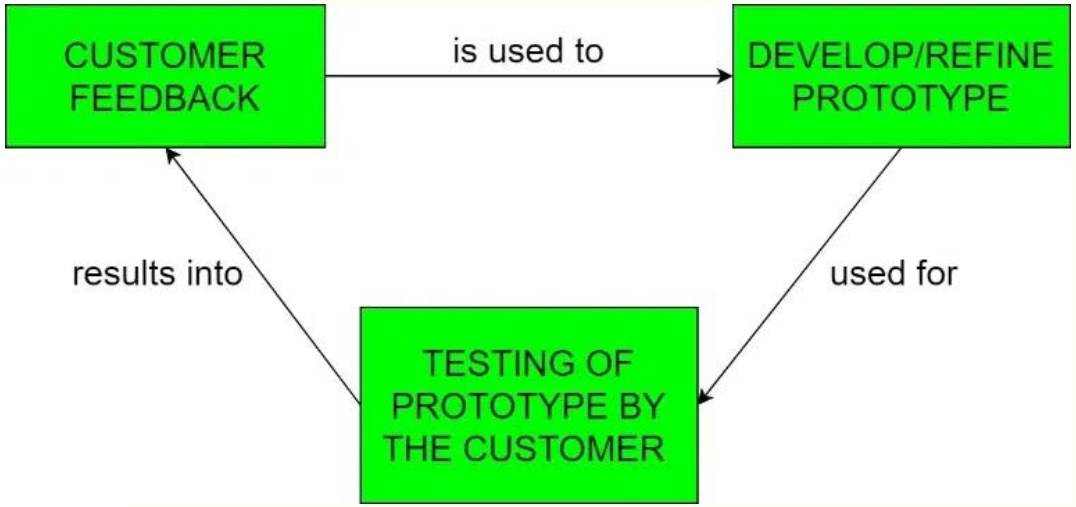
Prototyping Model

- Prototyping Model is used when the customers do not know the exact project requirements beforehand
- Communication : Prototyping model starts with communication
- Quick Plan : A prototyping iteration is planned quickly, and modeling occurs
- The quick design leads to the construction of a prototype. The prototype is deployed and evaluated by stakeholders

Prototyping Model



Some of the examples of prototypes are wireframes, slides, landing pages, working models, an interactive frontend, and videos.



Advantages of Prototype Model

- This model is flexible in design.
- It is easy to detect errors.
- We can find missing functionality easily.
- There is scope of refinement, it means new requirements can be easily accommodated.
- It can be reused by the developer for more complicated projects in the future.
- It ensures a greater level of customer satisfaction and comfort.
- It is ideal for online system.
- It helps developers and users both understand the system better.
- Integration requirements are very well understood and deployment channels are decided at a very early stage.
- It can actively involve users in the development phase.

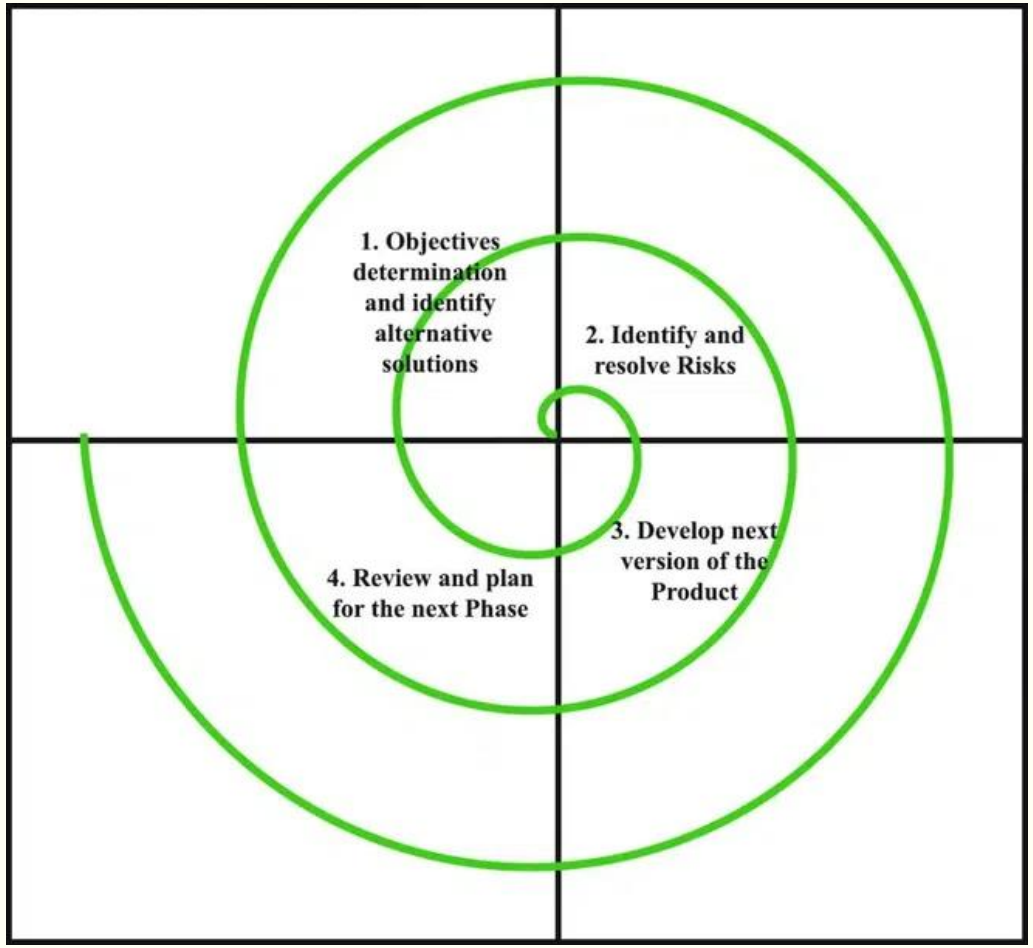
Disadvantages of Prototype Model

- This model is costly.
- It has poor documentation because of continuously changing customer requirements.
- There may be too much variation in requirements.
- Customers sometimes demand the actual product to be delivered soon after seeing an early prototype.
- There may be sub-optimal solutions because of developers in a hurry to build prototypes.
- Customers may not be satisfied or interested in the product after seeing the initial prototype.
- There is certainty in determining the number of iterations.
- There may be incomplete or inadequate problem analysis.
- There may increase the complexity of the system.

The Spiral Model

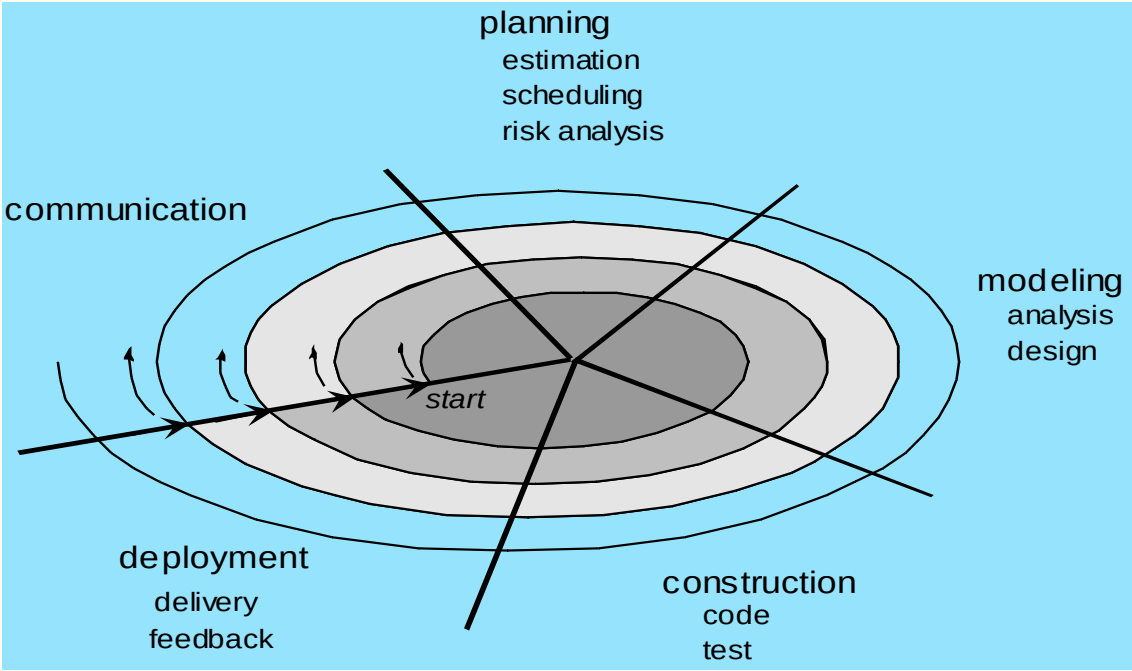
- Spiral Model is a risk-driven software development process model, meaning that the focus is on managing risk through multiple iterations of the software development process.
- It is a combination of waterfall model and iterative model.
- Spiral Model helps to adopt software development elements of multiple process models for the software project based on unique risk patterns ensuring efficient development process.
- The spiral model in software engineering was first mentioned by Barry Boehm in his 1986 paper.
- A Spiral model in software engineering is used when project is large, expensive and complicated projects

The Spiral Model



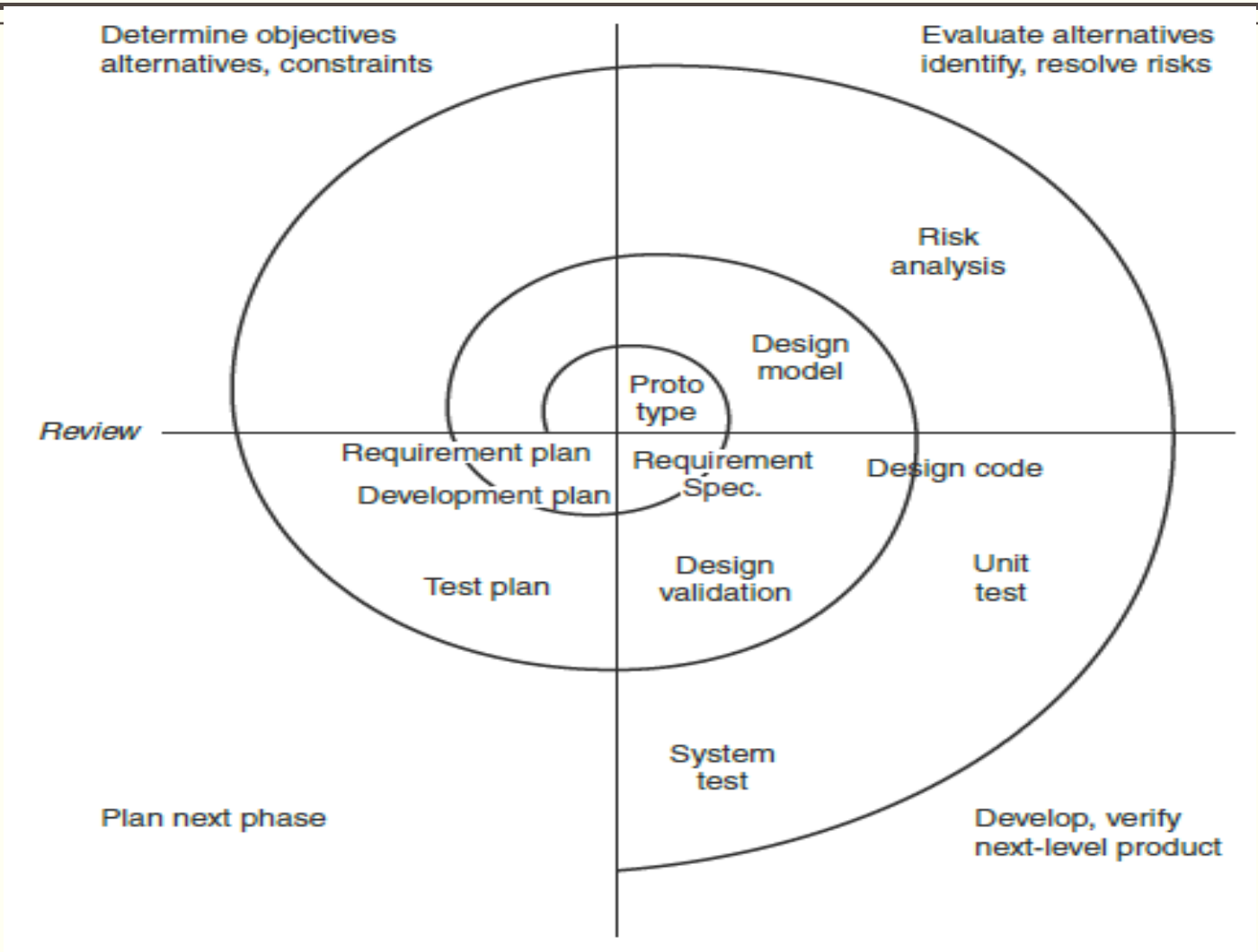
- The next iteration of the spiral begins with a new planning phase, based on the results of the evaluation.
- The Radius of the spiral at any point represents the expenses(cost) of the project so far, and the angular dimension represents the progress made so far in the current phase.

The Spiral Model



- The examples of spiral model are -
  - Microsoft used it to develop early versions of Windows.
  - The Gantt chart software was also made using spiral model.
  - Game development is another industry who uses spiral model to develop the games.

Spiral Model



Advantages & Disadvantages of Spiral Model

Advantages-

- High amount of risk analysis
- Useful for large and mission-critical projects.
- Changing requirements can be accommodated.

Disadvantages-

- Can be a costly model to use.
- Risk analysis needed highly particular expertise
- Doesn't work well for smaller projects.
- Large number of intermediate stages requires excessive documentation.

Advantages & Disadvantages of Evolutionary model

Advantages -

- User can easily understood the system and experiment with partially developed system.
- Useful in identifying requirement.
- It reduces the chance of error by testing core model multiple times .

Disadvantage -

- If the problem is not easy to divide into module then this module cannot be implemented correctly .

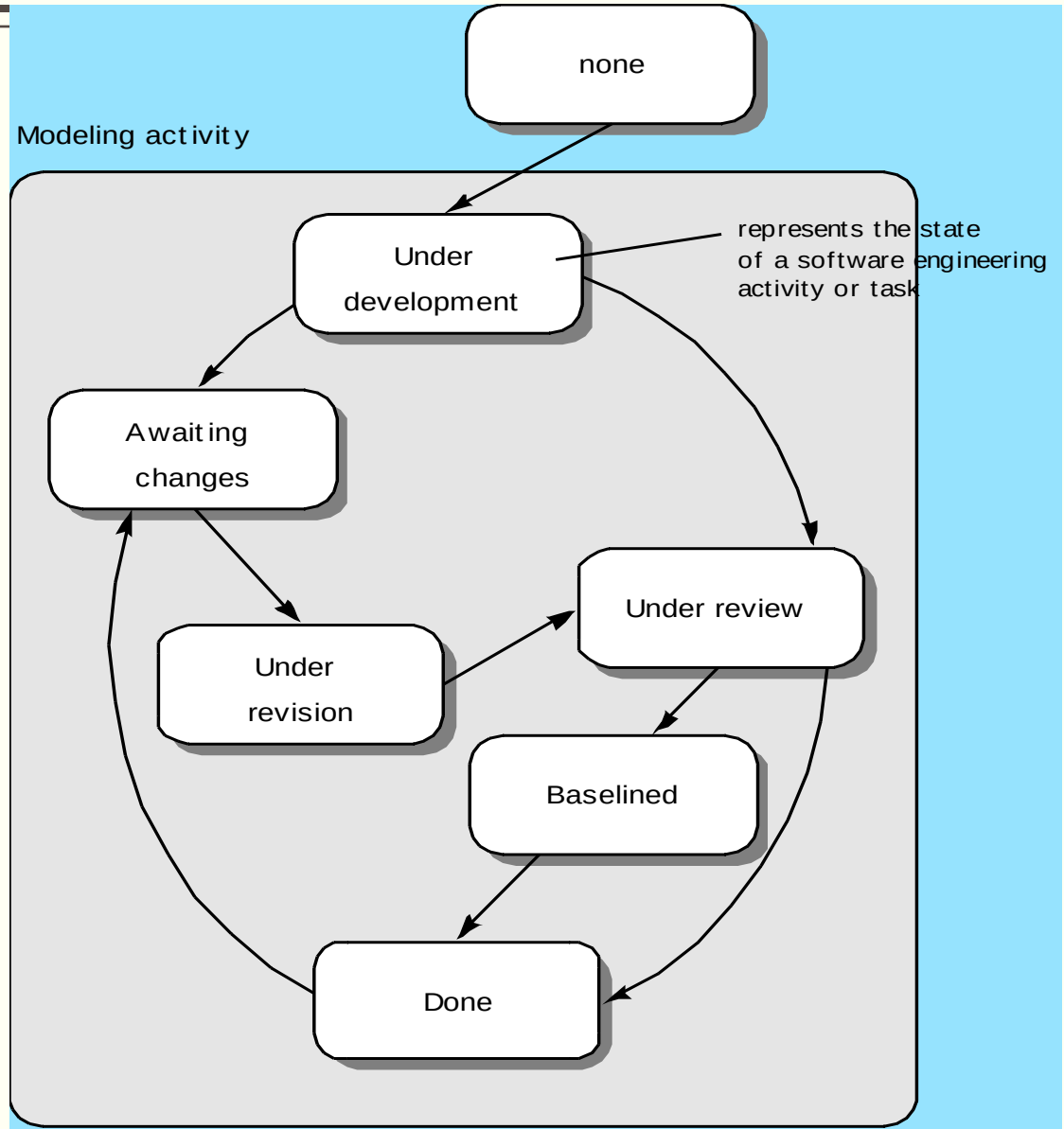
Weakness of Evolutionary model

- The first concern is that prototyping [and other more sophisticated evolutionary processes] poses a problem to project planning because of the uncertain number of cycles required to construct the product . . .
- Second, evolutionary software processes do not establish the maximum speed of the evolution.
  - the evolutions occur too fast----process will fall into chaos.
  - speed is too slow ---- productivity could be affected .
- Third, [evolutionary] software processes should be focused on flexibility and extensibility rather than on high quality. This assertion sounds scary.

- The intent of evolutionary models is to develop high-quality software in an iterative or incremental manner

Concurrent Model

- The concurrent model is a methodology in which multiple phases of the software development process are performed simultaneously (concurrently).
- The main goal of the concurrent development model is to produce a final product that meets the customer's requirements while minimizing the overall development time and cost.



Advantages & Disadvantages of Concurrent Model

Advantages -

- This model is applicable to all types of software development processes.
- It is easy for understanding and use.
- It gives immediate feedback from testing.
- It provides an accurate picture of the current state of a project.

Disadvantages -

- It needs better communication between the team members. This may not be achieved all the time.
- It requires to remember the status of the different activities.

Agile Development

When to use the Agile Model?

- When frequent changes are required.
- When a highly qualified and experienced team is available.
- When a customer is ready to have a meeting with a software team all the time.
- When project size is small.

Agile Development

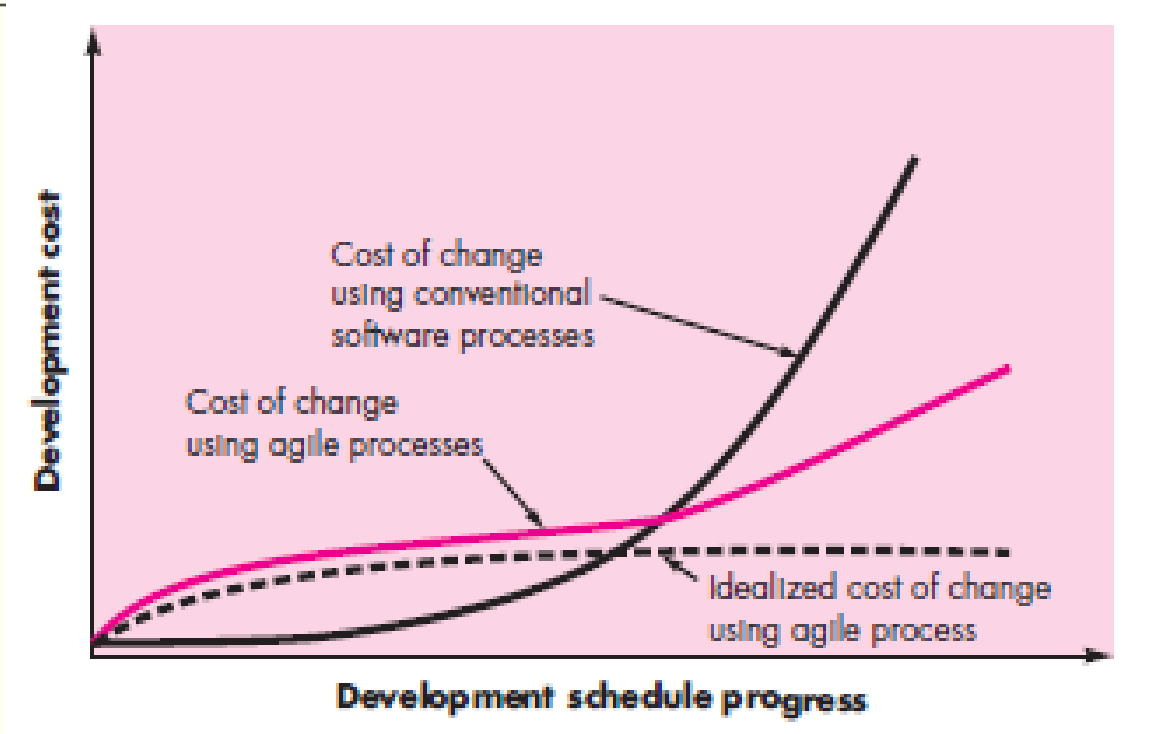
- The meaning of Agile is swift or versatile.
- "**Agile process model**" refers to a software development approach based on iterative development.
- Agile methods break tasks into smaller iterations, or parts do not directly involve long term planning.
- The main aim of Agility is customer satisfaction .
- Agile team supports -
  - Changes in the software being built,
  - Changes to the team members,
  - Changes because of new technology,
  - Changes of all kinds that may have an impact on the product they build or the project that creates the product.

Agile Model

Fig. Agile Model

- It emphasizes rapid delivery of operational software and deemphasizes the importance of intermediate work products (not always a good thing);
- It adopts the customer as a part of the development team and works to eliminate the "us and them" attitude that continues to pervade many software projects.
- It recognizes that planning in an uncertain world has its limits and that a project plan must be flexible.

Agility and Cost to Change



- The cost of change increases nonlinearly as a project progresses
- It is relatively easy to accommodate a change when a software team is gathering requirements (early in a project).
- Costs escalate quickly, and the time and cost required to ensure that the change is made without unintended side effects is nontrivial.

Agile Process – Extreme Programming (XP)

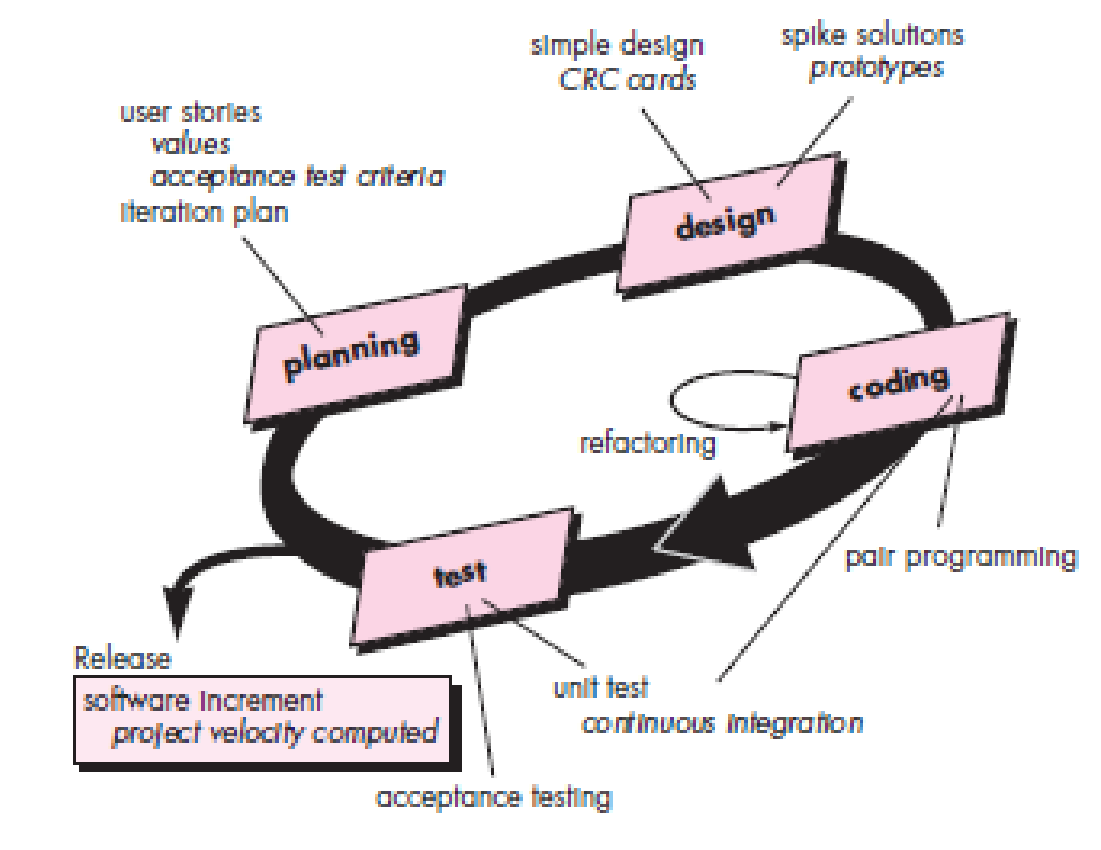
- The most widely used approach to agile software development

1. **XP Values**
   - A set of five *values* that establish a foundation for all work performed as part of XP—**communication, simplicity, feedback, courage, and respect.**
   - Establishing required features and functions for the software can be achieved with effective **communication** between software engineers and other stakeholders.
   - To achieve **simplicity**, XP restricts developers to design only for immediate needs, rather than consider future needs.
   - **Feedback** is derived from three sources: the implemented software itself, the customer, and other software team members.
   - Strict adherence to certain XP practices demands **courage**.
   - By following each of these values, the agile team inculcates **respect** among it members, between other stakeholders and team members, and indirectly, for the software itself.

Agile Process – Extreme Programming (XP)

2. The XP Process

Extreme Programming uses an object-oriented approach and encompasses a set of rules and practices that occur within the context of four framework activities: **planning, design, coding, and testing.**



Applications of Extreme Programming (XP)

- Small Projects
- Projects involving new technology or Research projects
- Web development projects
- Projects with tight deadlines
- Projects with rapidly changing requirements

Other Agile Process Models

- Adaptive Software Development (ASD)
- Scrum
- Dynamic Systems Development Method (DSDM)
- Crystal
- Feature Drive Development (FDD)
- Lean Software Development (LSD)
- Agile Modelling (AM)
- Agile Unified Process (AUP)

Scrum

- Scrum is an agile software development method
- Scrum principles are consistent with the agile manifesto and are used to guide development activities within a process that incorporates the following framework activities: requirements, analysis, design, evolution, and delivery.
- Within each framework activity, work tasks occur within a process pattern called a sprint. The work conducted within a sprint is adapted to the problem at hand and is defined and often modified in real time by the Scrum team.
- The overall flow of the Scrum process is illustrated in Figure.

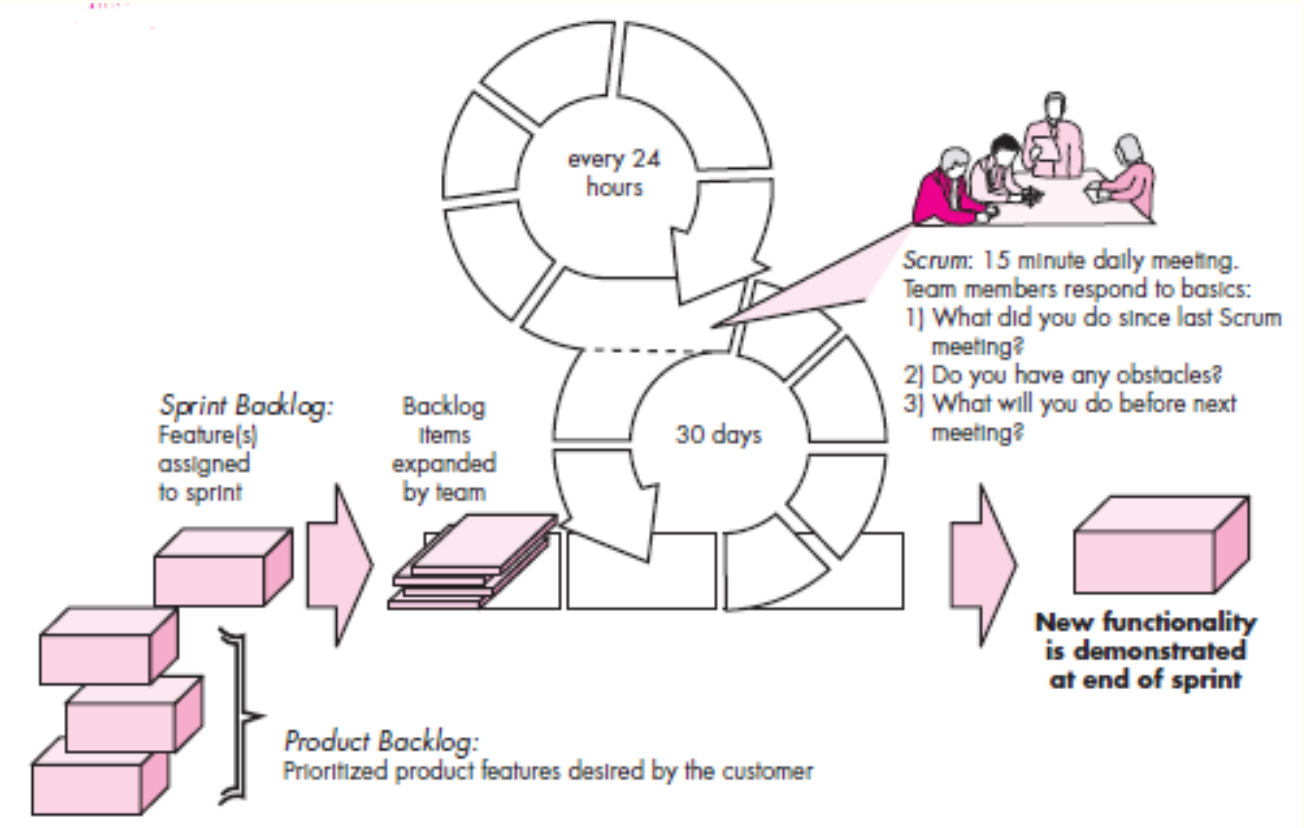


Figure: Scrum process flow

Scrum

- Scrum emphasizes the use of a set of software process patterns that have proven effective for projects with tight timelines, changing requirements, and business criticality. Each of these process patterns defines a set of development actions:
  - *Backlog*—a prioritized list of project requirements or features that provide business value for the customer. Items can be added to the backlog at any time. The product manager assesses the backlog and updates priorities as required.
  - *Sprints*—consist of work units that are required to achieve a requirement defined in the backlog that must be fit into a predefined time-box.
  - *Scrum meetings*—are short (typically 15 minutes) meetings held daily by the Scrum team.
  - *Demos*—deliver the software increment to the customer so that functionality that has been implemented can be demonstrated and evaluated by the customer.

- The Scrum process patterns enable a software team to work successfully in a world where the elimination of uncertainty is impossible.

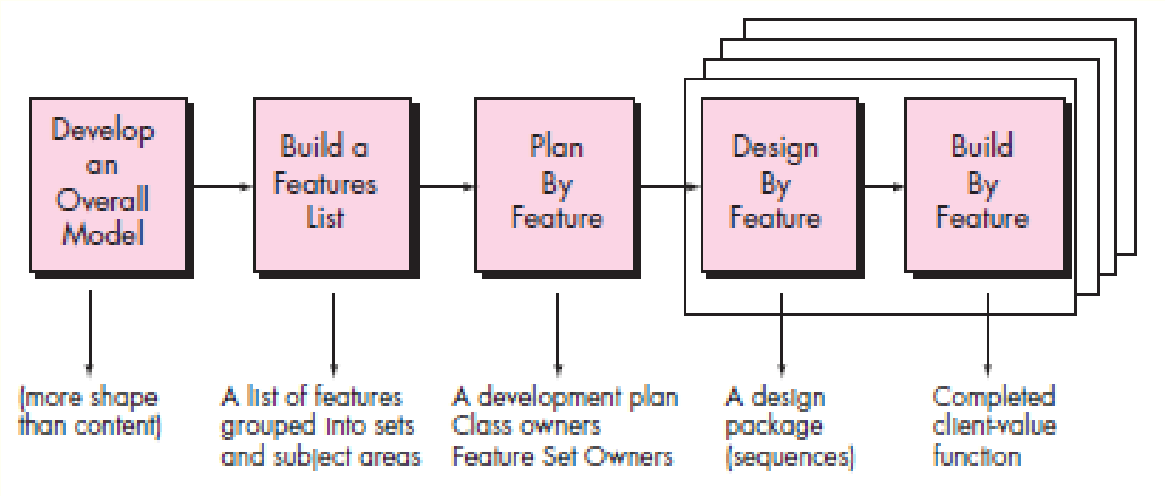
Feature Driven Development (FDD)

- Like other agile approaches, FDD adopts a philosophy that
  - (1) emphasizes collaboration among people on an FDD team;
  - (2) manages problem and project complexity using feature-based decomposition followed by the integration of software increments, and
  - (3) communication of technical detail using verbal, graphical, and text-based means.

- FDD emphasizes software quality assurance activities by encouraging an incremental development strategy, the use of design and code inspections, the application of software quality assurance audits, the collection of metrics, and the use of patterns.

- In the context of FDD, a *feature* "is a client-valued function that can be implemented in two weeks or less".

Feature Driven Development (FDD)



- The FDD approach defines five "collaborating" framework activities as shown in Figure.
- FDD provides greater emphasis on project management guidelines and techniques than many other agile methods.
- As projects grow in size and complexity, ad hoc project management is often inadequate.
- It is essential for developers, their managers, and other stakeholders to understand project status—what accomplishments have been made and problems have been encountered. If deadline pressure is significant, it is critical to determine if software increments (features) are properly scheduled.
- To accomplish this, FDD defines six milestones during the design and implementation of a feature: "design walkthrough, design, design inspection, code, code inspection, promote to build"

	Software Process Model	Advantages	Disadvantages
Plan Driven	• Waterfall • Incremental Development • Iterative development • Spiral Development • Prototype Model • Rapid Application Development	• Suitable for large systems and teams. • Handles highly critical systems effectively. • Appropriate for stable development environment. • Require experienced personnel at the beginning. • Success achieved through structure and order.	• Longer length in each iteration or increment. • Cannot accommodate changes any time. • Lack of user involvement throughout the life cycle of the product. • Costly for the dynamic development environment. • Assume that, future changes will not occur.
Agile	• Scrum model • Extreme Programming (XP) • Dynamic System Development Method • Kanban • Feature Driven Development	• Suitable for small to medium systems and teams. • Can accommodate changes at any time. • Effective for the dynamic development environment. • Required expert agile personnel throughout the life cycle. • Success achieved through freedom and chaos.	• Not suitable for large systems (except FDD). • Shorter length in each iteration. • Can accommodate changes at any time. • Costly for the stable development environment. • Assume that, frequent future changes will occur.

Comparison between plan-driven and agile development

Perspective	Agile methods	Plan-driven methods
Primary objective	Rapid value	High assurance
Developers	Agility Knowledgeability Collocation Collaboration	Plan-orientation Skills adequacy Accessing external knowledge
Customers	Dedication Knowledgeability Collocation Collaboration Representation Empowering	Accessing to knowledge Collaboration Representation Empowering
Requirements	Highly emerging Rapid changing	Knowable early Highly stable
Size	Smaller (teams and products)	Larger (teams and products)
Architecture	Design for current requirements	Design for current and predictable requirements
Refactoring	Inexpensive	Expensive

THANK YOU !!

K K Wagh